

AWS

COOKBOOK

REAL SOLUTIONS FOR REAL ENGINEERS

BUILD | DEPLOY | SECURE | AUTOMATE | TROUBLESHOOT | SCALE



PRACTICAL
RECIPES



REAL-WORLD
SOLUTIONS



TROUBLESHOOTING
GUIDES



SECURITY BEST
PRACTICES



PRODUCTION
READY



EC2



S3



RDS



Lambda



VPC



CloudWatch



IAM



Route 53



EKS



ECS

SIMPLE RECIPES. POWERFUL RESULTS.

FROM
INFRASTRUCTURE
TO PRODUCTION



LEARN



PRACTICE



SOLVE



GROW

Recipe 01

AWS IDENTITY & ACCESS MANAGEMENT (IAM)

Give an Application AWS Access Without Hardcoding Credentials

Use IAM roles instead of access keys embedded in code.
Follow least privilege and use temporary credentials.

PROBLEM

- Your application needs to access AWS services (e.g. S3, DynamoDB) but you don't want to hardcode access keys in the code.
- Hardcoded credentials are insecure and can be leaked.
- You want a secure, manageable and production-ready solution.

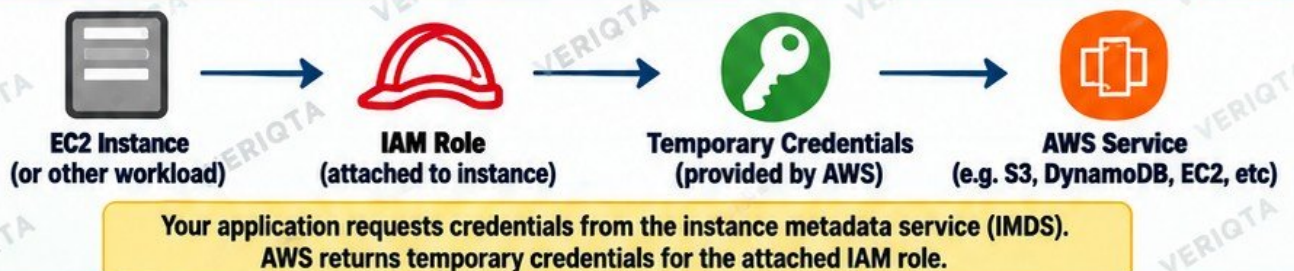
SOLUTION

- Attach an IAM role to the workload (EC2, ECS, Lambda, etc).
- AWS provides temporary credentials automatically.
- Use least privilege permissions.
- No access keys in your code.
- Credentials rotate automatically.

KEY BENEFITS

- ✓ More secure
- ✓ No hardcoded credentials
- ✓ Automatic credential rotation
- ✓ Easier permission management
- ✓ Works with EC2, Lambda, ECS, EKS and more
- ✓ Follows AWS best practices

HOW IT WORKS



EXAMPLE: ATTACH IAM ROLE TO EC2

- 1 Create an IAM role with the required permissions (e.g. S3 read access).
- 2 Attach the role to your EC2 instance.
- 3 Use the AWS SDK in your application. The SDK automatically uses the temporary credentials.
- 4 No access keys needed in your code.

VERIFY IT WORKS

On the EC2 instance, check the temporary credentials:

```
curl http://169.254.169.254/latest/meta-data/iam/security-credentials/
curl http://169.254.169.254/latest/meta-data/iam/security-credentials/<role-name>
```

You will see temporary credentials (AccessKeyId, SecretAccessKey, Token, Expiration).

CLASSIC FAILURE: ACCESS DENIED

An error occurred (AccessDenied) when calling the ListBuckets operation: User: arn:aws:sts::123456789012:assumed-role/MyRole/i-0abc123 is not authorized to perform s3:ListBuckets on resource: arn:aws:s3::*

TROUBLESHOOTING STEPS

- Check the IAM role has the correct permissions.
- Verify the role is attached to the instance.
- Confirm you are calling the correct AWS region.
- Make sure you are using the AWS SDK and not hardcoded keys.
- Use least privilege and allow only the required actions.

BEST PRACTICES

- ✓ Use IAM roles instead of access keys.
- ✓ Follow least privilege principle.
- ✓ Use AWS managed policies where possible.
- ✓ Rotate and manage permissions centrally.
- ✓ Do not log or expose temporary credentials.
- ✓ Use condition keys to further restrict access.
- ✓ Regularly review and audit permissions.

REAL WORLD USE CASES

- ✓ EC2 accessing S3 for application files.
- ✓ ECS tasks accessing DynamoDB.
- ✓ Lambda reading from SQS.
- ✓ EKS workloads accessing AWS APIs.

ENGINEERS
BUILD A
BETTER
TOMORROW

Recipe 02

Get Into a Private EC2 Instance Without Opening SSH to the Internet

Use AWS Systems Manager Session Manager for secure, auditable and firewall-friendly access to your instances.

THE PROBLEM

- Your EC2 instance is in a private subnet with no public IP.
- You need to connect to the instance for administration.
- You do not want to open SSH (port 22) to the internet.
- You need a secure and auditable way to access the instance.

THE SOLUTION

- Use AWS Systems Manager Session Manager.
- Attach the AmazonSSMManagedInstanceCore IAM role to the instance.
- Ensure the SSM Agent is running on the instance.
- The instance connects to AWS over HTTPS (no inbound ports required).
- You get secure shell access through the AWS Console or AWS CLI.

KEY BENEFITS

- ✓ No public IP required
- ✓ No open SSH ports
- ✓ More secure
- ✓ Works in private subnets
- ✓ Session logging and auditing
- ✓ No need to manage SSH keys
- ✓ Follows AWS best practices
- ✓ Works with EC2, on-premises servers and other workloads

HOW IT WORKS



No Inbound Ports Needed

- ✓ Instance in private subnet
- ✓ No public IP
- ✓ No SSH (port 22)
- ✓ Outbound HTTPS to AWS
- ✓ SSM Agent must be running
- ✓ IAM role attached

RECIPE STEPS

- 1 Create or use the IAM role AmazonSSMManagedInstanceCore.
- 2 Attach the role to your EC2 instance.
- 3 Ensure the SSM Agent is installed and running (most Amazon Linux and Ubuntu AMIs include it).
- 4 Make sure the instance has network access to SSM endpoints (via internet, NAT Gateway or VPC endpoints).
- 5 Connect using the AWS Console or AWS CLI.

CONNECT USING AWS CLI

```
aws ssm start-session \
  --target i-0123456789abcdef0
```

- Replace the instance ID with your own.
- You will get a shell session on the instance.
- Works for Linux and Windows (PowerShell/RDP).

TROUBLESHOOTING: INSTANCE NOT MANAGED

- Check that the IAM role is attached to the instance.
- Verify the SSM Agent is running: `sudo systemctl status amazon-ssm-agent`
- Ensure the instance has network access to SSM endpoints (or internet through a NAT Gateway).
- Check the instance is in a supported region.
- Verify no firewall or proxy is blocking outbound HTTPS (port 443).
- Check the instance appears in Systems Manager → Managed Instances.

BEST PRACTICES

- ✓ Use IAM roles instead of access keys.
- ✓ Keep instances in private subnets.
- ✓ Use VPC endpoints for SSM in fully private environments.
- ✓ Enable session logging (to S3 or CloudWatch Logs).
- ✓ Use IAM policies to restrict who can start a session.
- ✓ Regularly review and audit session logs.
- ✓ Do not install or enable SSH unless absolutely required.

Recipe 03

- ✓ SECURE
- ✓ SCALABLE
- ✓ PRODUCTION
- ✓ READY

Put an Application Server in a Private Subnet but Keep Outbound Internet Access

Build the practical public/private subnet pattern.

Private EC2 → route table → NAT Gateway → Internet Gateway.

THE PROBLEM

- You want your application server in a private subnet (not accessible from the internet).
- The server needs outbound internet access to download packages, updates and other dependencies.
- The server can run, but cannot reach package repositories (e.g. yum, apt, pip).
- You want a secure and scalable solution.

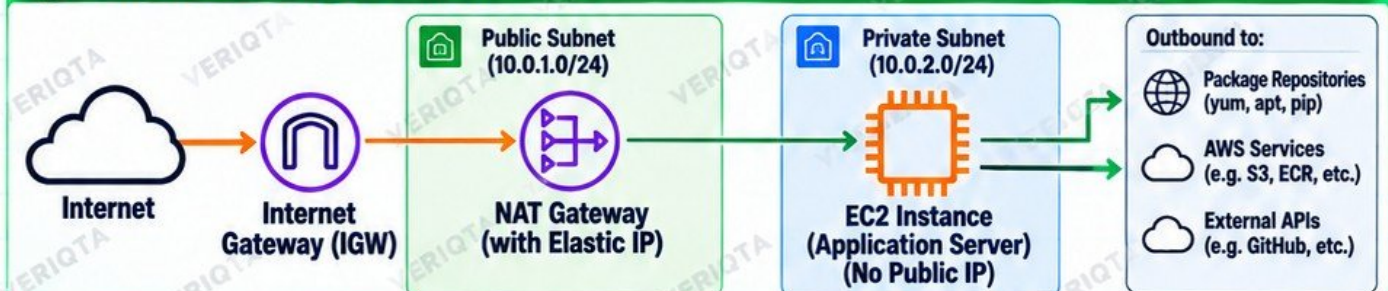
THE SOLUTION

- Place the application server in a private subnet.
- Use a NAT Gateway in a public subnet for outbound internet access.
- Route private subnet traffic (0.0.0.0/0) to the NAT Gateway.
- The NAT Gateway uses the Internet Gateway to reach the internet.
- No inbound access to the instance is required.
- The instance can download packages, access external APIs and get updates securely.

KEY BENEFITS

- ✓ Instance stays private (no public IP).
- ✓ Outbound internet access for updates and dependencies.
- ✓ Reduced attack surface (no open SSH).
- ✓ Works for any application (Linux or Windows).
- ✓ Scalable and production ready.
- ✓ Follows AWS best practices.

ARCHITECTURE DIAGRAM



ROUTE TABLES

Route Table	Destination	Target
Public Subnet Route Table	0.0.0.0/0	Internet Gateway (IGW)
Private Subnet Route Table	0.0.0.0/0	NAT Gateway
VPC Local	10.0.0.0/16	local

TROUBLESHOOTING SCENARIO

- "The server can run, but cannot reach package repositories."
- Common causes:**
- No route to 0.0.0.0/0 in private subnet route table.
 - NAT Gateway not in a public subnet.
 - NAT Gateway has no Elastic IP.
 - Security Group or NACL blocking outbound traffic.
 - DNS resolution not working.
 - Instance using the wrong route table.
 - No internet access from NAT Gateway (due to missing IGW route).

RECIPE STEPS

- 1 Create a VPC with public and private subnets.
- 2 Create and attach an Internet Gateway (IGW).
- 3 Create a NAT Gateway in a public subnet and assign an Elastic IP.
- 4 Update the private subnet route table: 0.0.0.0/0 → NAT Gateway.
- 5 Launch your EC2 instance in the private subnet (with no public IP).
- 6 Ensure the instance has internet access (via NAT Gateway).
- 7 Test by downloading a package from the internet.

VERIFY IT WORKS

```
# Check internet access
curl -I https://amazonlinux.com
sudo yum update -y # (Amazon Linux)
sudo apt update # (Ubuntu)
```

BEST PRACTICES

- ✓ Keep instances in private subnets.
- ✓ Use NAT Gateway (not NAT instance) for high availability.
- ✓ Use VPC endpoints for AWS services (S3, ECR, etc.) to reduce NAT cost.
- ✓ Monitor NAT Gateway metrics and costs.
- ✓ Use least privilege security groups.

**PRIVATE
BUT CONNECTED
THAT'S SMART
INFRASTRUCTURE**

Recipe 04

 TROUBLESHOOT
UNDERSTAND
SOLVE
DON'T GUESS

Find Why an EC2 Instance Cannot Reach Another AWS Resource

A network troubleshooting recipe. Check DNS, routes, security groups, network ACLs, ports and Reachability Analyzer.

THE PROBLEM

- Your EC2 instance cannot reach another AWS resource (e.g. RDS, S3, another EC2, or a service).
- The application times out or shows "Connection refused".
- You are not sure if it is DNS, routing, security group, NACL or a blocked port.
- You want a systematic way to find and fix the issue.

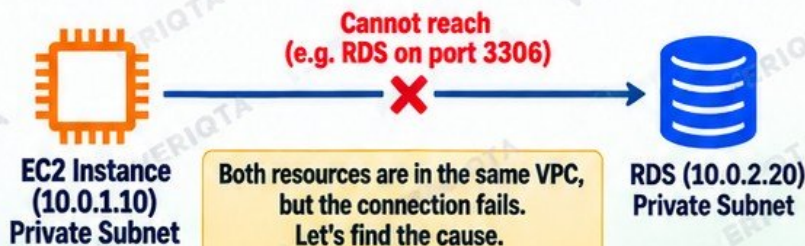
THE SOLUTION

- Follow a step-by-step network troubleshooting process.
- Check DNS resolution, routing, security groups, NACLs and ports.
- Use VPC Reachability Analyzer to identify the exact blocking point.
- Fix the real cause instead of guessing or randomly changing firewall rules.
- Verify the connection after each fix.

KEY TAKEAWAYS

- ✓ Most connectivity issues are caused by DNS, routes, security groups, NACLs or blocked ports.
- ✓ Use tools and evidence, not guesswork.
- ✓ Reachability Analyzer saves time.
- ✓ Both inbound and outbound rules must allow the traffic.
- ✓ NACLs can silently block traffic even when security groups allow it.
- ✓ Always verify after making changes.

COMMON SCENARIO



POSSIBLE DESTINATIONS

- RDS / Aurora
- S3 (via VPC endpoint or internet)
- Another EC2 instance
- EKS / ECS service
- Application/Network Load Balancer
- Lambda (via VPC endpoint)
- Any AWS service

TROUBLESHOOTING STEPS

- 1 Check DNS Resolution – Can the instance resolve the hostname?
- 2 Check Security Groups – Is outbound allowed from source and inbound allowed to destination?
- 3 Check Network ACLs – Make sure NACLs allow the traffic.
- 4 Check Routing – Is there a route to the destination?
- 5 Check the Port – Is the service listening on the expected port?
- 6 Use Reachability Analyzer – Identify the exact blocking component.
- 7 Verify Fix – Test the connection again.

USEFUL COMMANDS (ON EC2)

```
# Test DNS resolution
nslookup rds-endpoint.amazonaws.com
dig rds-endpoint.amazonaws.com

# Test network connectivity
telnet 10.0.2.20 3306
nc -zv 10.0.2.20 3306

# Test HTTPS (e.g. S3, API)
curl -I https://s3.amazonaws.com

# Check routing table
ip route

# Check listening ports
sudo ss -tulnp | grep 3306
```

VPC REACHABILITY ANALYZER

- Open VPC → Reachability Analyzer.
- Select the source (EC2) and destination (e.g. RDS).
- Run the analysis.
- It shows the exact component that is blocking the traffic (e.g. security group, NACL, or route).

COMMON CAUSES & FIXES

Cause	Fix
DNS resolution fails	Check VPC DNS settings and hostname.
Security group blocks traffic	Allow correct inbound/outbound rules.
NACL blocks traffic	Allow the required port in NACL.
No route to destination	Check route tables and VPC peering/transit gateway.
Service not listening	Verify the application is running on the port.
Using public endpoint from private subnet	Use VPC endpoint or NAT Gateway.

Recipe 05

Put an Application Behind an HTTPS Load Balancer

- ✓ SECURE
- ✓ RELIABLE
- ✓ SCALABLE
- ✓ PRODUCTION READY

Use Route 53 → Application Load Balancer → Target Group → EC2.
Redirect HTTP to HTTPS, attach a certificate, configure health checks, and diagnose unhealthy targets.

THE PROBLEM

- Your application is running on EC2, but users connect directly to the instance.
- Traffic is not encrypted (HTTP).
- There is no single, reliable endpoint for your application.
- You need a secure, scalable, production-ready setup.

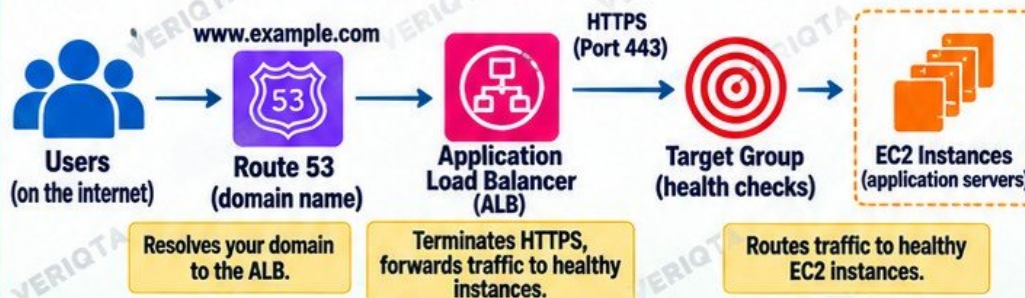
THE SOLUTION

- Use an Application Load Balancer (ALB) with HTTPS.
- Attach an SSL/TLS certificate (AWS Certificate Manager).
- Redirect HTTP (port 80) to HTTPS (port 443).
- Register your EC2 instances in a target group.
- Configure health checks.
- Use Route 53 for a friendly domain name.

KEY BENEFITS

- ✓ Encrypts traffic (HTTPS).
- ✓ Single, stable endpoint.
- ✓ Distributes traffic across healthy instances.
- ✓ Automatically removes unhealthy instances.
- ✓ Easier scaling and high availability.
- ✓ Integrates with AWS services (Route 53, ACM, Auto Scaling).
- ✓ Follows AWS best practices.

ARCHITECTURE FLOW



Key Components

- Route 53 – DNS
- ACM – SSL/TLS certificate
- ALB – HTTPS and routing
- Target Group – health checks
- EC2 – application servers
- Security Groups – control traffic
- Auto Scaling – (optional)

RECIPE STEPS

- 1 Request or import an SSL/TLS certificate in ACM.
- 2 Create an ALB (internet-facing) with an HTTPS listener (443).
- 3 Add an HTTP listener (80) and redirect to HTTPS.
- 4 Create a target group (e.g. port 80) and configure health checks (e.g. / or /health).
- 5 Register your EC2 instances with the target group.
- 6 Update your Route 53 DNS record to point to the ALB.
- 7 Test access using <https://your-domain.com>.

HEALTH CHECK EXAMPLE

Protocol	HTTP or HTTPS
Path	/ or /health
Port	80 (or your app port)
Interval	30 seconds
Timeout	5 seconds
Healthy threshold	2
Unhealthy threshold	2

VERIFY IT WORKS

- 1 Open <https://your-domain.com> in a browser.
- ✓ Check that the certificate is valid (no browser warning).
- ✓ Confirm traffic reaches your application.
- ✓ In the ALB console, target status should be Healthy.

TROUBLESHOOTING: UNHEALTHY TARGETS

Symptom	Possible Cause	Fix
Target shows Unhealthy	Application not running or wrong port	Check application logs, port and security group.
Health check timeout	Security group blocking ALB → EC2	Allow ALB security group to your instance port.
HTTP 301/302 instead of 200	Health check path redirects	Use a path that returns 200 (e.g. /health).
Cannot reach application	DNS, SG, or listener issue	Check Route 53, listener rules and security groups.
SSL certificate error	Wrong certificate or domain	Use correct ACM certificate for your domain.

COMMON MISTAKES

- Forgetting to redirect HTTP to HTTPS.
- Using the wrong health check path.
- Blocking ALB traffic in security groups.
- Registering instances in the wrong target group.
- Using a self-signed certificate in production.
- Not verifying target health after deployment.

BEST PRACTICES

- ✓ Use ACM for certificates (free and auto-renewal).
- ✓ Redirect all HTTP traffic to HTTPS.
- ✓ Use meaningful health check endpoints.
- ✓ Use security groups (not NACLs) for most cases.
- ✓ Combine with Auto Scaling for high availability.
- ✓ Monitor with CloudWatch and set alarms.
- ✓ Use a custom domain with Route 53.

GOOD
INFRASTRUCTURE
BUILDS
GREAT THINGS

Recipe 06

Automatically Replace Failed Application Servers

Build an Auto Scaling Group behind an ALB. Show health checks, minimum/desired/maximum capacity, instance replacement, scaling policy, and what happens when an instance dies.

✓ SCALE
✓ HEAL
✓ AUTOMATE
✓ STAY AVAILABLE

LEARN
BUILD
DEPLOY
GROW

THE PROBLEM

- An application server fails (crash, OS issue, or hardware failure).
- Users should not be affected.
- You want automatic replacement and the ability to scale based on demand.
- You don't want to manage servers manually.

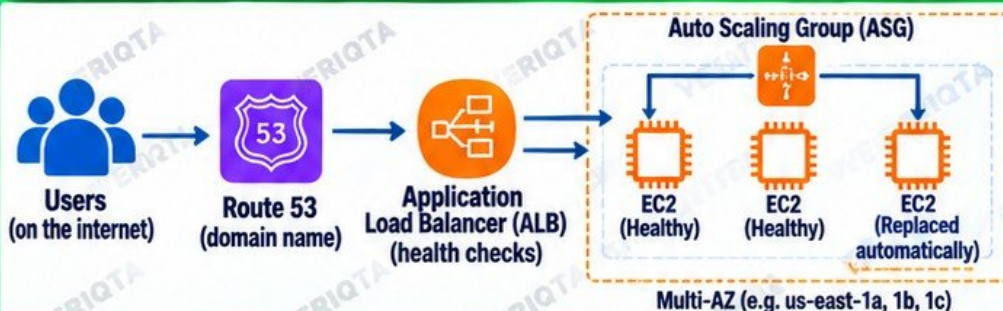
THE SOLUTION

- Use an Auto Scaling Group (ASG) behind an Application Load Balancer (ALB).
- Enable health checks (ALB and EC2).
- Configure minimum, desired and maximum capacity.
- Use scaling policies (e.g. CPU-based).
- When an instance fails, ASG automatically launches a new one.
- Traffic is only sent to healthy instances via the ALB.

KEY BENEFITS

- ✓ Automatic instance replacement.
- ✓ High availability and fault tolerance.
- ✓ Scales based on demand.
- ✓ No manual intervention.
- ✓ Works across multiple Availability Zones.
- ✓ Only healthy instances receive traffic.
- ✓ Follows AWS best practices.

ARCHITECTURE DIAGRAM



Key Components

- Route 53 – DNS
- Application Load Balancer
- Target Group (health checks)
- Auto Scaling Group (ASG)
- Launch Template (or Launch Configuration)
- Security Groups
- EC2 Instances (multi-AZ)
- CloudWatch (monitoring)

ASG CONFIGURATION

Minimum capacity	2	(always keep at least 2)
Desired capacity	2	(normal running count)
Maximum capacity	6	(scale up to 6 if needed)
Health check type	ALB	(or EC2)
Scaling policy	CPU > 70% → add instance CPU < 30% → remove instance	

WHAT HAPPENS WHEN AN INSTANCE DIES?

- 1 ALB health check detects the instance is unhealthy.
- 2 ALB stops sending traffic to the failed instance.
- 3 ASG terminates the unhealthy instance.
- 4 ASG automatically launches a new instance.
- 5 The new instance is registered with the target group.
- 6 Traffic resumes to the new healthy instance.
- 7 Your application stays available.

USEFUL AWS CLI COMMANDS

```

# Check ASG instances
aws autoscaling describe-auto-scaling-groups --auto-scaling-group-names my-asg

# Check instance health in target group
aws elbv2 describe-target-health \
  --target-group-arn <target-group-arn>

# View scaling activities
aws autoscaling describe-scaling-activities \
  --auto-scaling-group-name my-asg
  
```

SCALING POLICY EXAMPLE (CPU-BASED)

- If average CPU > 70% for 5 minutes → Add 1 instance
- If average CPU < 30% for 10 minutes → Remove 1 instance
- Use CloudWatch alarms to trigger the scaling policy.
- You can also scale based on memory, request count, or custom metrics.

COMMON ISSUES & FIXES

Issue	Possible Cause	Fix
Instance not replaced	Health checks not configured	Enable ALB health checks
New instance unhealthy	App not running / wrong port	Fix app config and security group
No scaling activity	CloudWatch alarm not set	Create and attach scaling policy
Instances in same AZ	Subnet configuration incorrect	Use multiple AZ subnets
ALB not routing traffic	Target group mismatch	Check target group and health status

BEST PRACTICES

- ✓ Use at least 2 Availability Zones.
- ✓ Use ALB health checks (more reliable than EC2 status checks).
- ✓ Set reasonable min/desired/max capacity.
- ✓ Use launch templates for consistent configuration.
- ✓ Monitor with CloudWatch and set scaling alarms.
- ✓ Test instance failure to verify automatic replacement.
- ✓ Keep instances in private subnets (use NAT for updates).

AUTOMATION
BUILDS RESILIENCE

Recipe 07

Make an S3 Bucket Private Without Breaking the Application

✓ PRIVATE
✓ SECURE
✓ CONTROLLED
✓ PRODUCTION READY

LEARN
BUILD
DEPLOY
GROW

Block public access while allowing only the required workload to retrieve objects. Use bucket policies, IAM permissions, encryption, and troubleshoot AccessDenied.

⚠ THE PROBLEM

- Your S3 bucket is accessible publicly (or someone can make it public).
- You want to block all public access but keep your application working.
- You need to allow only specific workloads (e.g. EC2, ECS, Lambda) to retrieve objects.
- You want encryption and proper IAM permissions.
- You're getting AccessDenied errors after making changes.

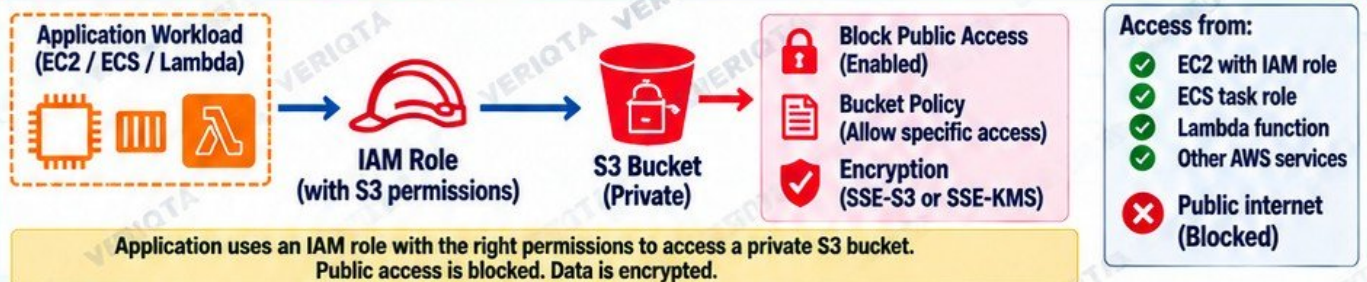
✅ THE SOLUTION

- Block all public access at the account and bucket level.
- Use IAM roles/policies for the application workload.
- Use a bucket policy to allow only the required access.
- Enable encryption (SSE-S3 or SSE-KMS).
- Test and verify access without making the bucket public.
- Use CloudTrail and logs to diagnose AccessDenied errors.

💡 KEY BENEFITS

- ✓ No public access to your data.
- ✓ Only authorized workloads can access objects.
- ✓ Data is encrypted at rest (and in transit).
- ✓ Reduced risk of data leaks.
- ✓ Meets security and compliance requirements.
- ✓ Works for EC2, ECS, Lambda and other AWS services.
- ✓ Follows AWS best practices.

⚙ ARCHITECTURE DIAGRAM



⚙ BUCKET POLICY EXAMPLE (Allow specific IAM role)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::123456789012:role/AppRole"
      },
      "Action": [
        "s3:GetObject"
      ],
      "Resource": "arn:aws:s3:::my-bucket/*"
    }
  ]
}
```

🔒 IAM POLICY EXAMPLE (Attach to the workload role)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::my-bucket",
        "arn:aws:s3:::my-bucket/*"
      ]
    }
  ]
}
```

⚙ ENCRYPTION OPTIONS

- SSE-S3 (AWS managed keys, simplest)
- SSE-KMS (Use a KMS key, more control)
- SSE-C (Customer provided keys)
- Always enforce encryption using bucket policy or S3 bucket settings.

🔧 TROUBLESHOOTING: ACCESSDENIED

Possible Cause	How to Check / Fix
Missing IAM permission	Check IAM policy on the role.
Bucket policy is too restrictive	Review bucket policy and principal.
Wrong region or bucket name	Verify region and correct bucket name.
Object does not exist	Check the object key and path.
Using wrong credentials	Confirm the workload is using the correct IAM role.
Public access blocked	This is expected. Allow only required access.
SSE-KMS permission missing	Check KMS key policy and IAM permissions.
VPC endpoint policy blocking	Review the VPC endpoint policy (if using one).

✅ VERIFICATION STEPS

- ✓ Confirm public access is blocked.
- ✓ Test access from the application (e.g. EC2 or Lambda).
- ✓ Use AWS CLI to verify: `aws s3 ls s3://my-bucket`
- ✓ Try downloading an object: `aws s3 cp s3://my-bucket/file.txt .`
- ✓ Check CloudTrail logs if access fails.
- ✓ Ensure encryption is enabled.
- ✓ Confirm no public bucket policy allows access.

📋 BEST PRACTICES

- ✓ Enable Block Public Access (at account and bucket level).
- ✓ Use least privilege IAM policies.
- ✓ Use bucket policies for fine-grained control.
- ✓ Encrypt all data at rest.
- ✓ Use VPC endpoints for S3 (from private subnets).
- ✓ Monitor access with CloudTrail.
- ✓ Regularly review permissions.
- ✓ Never make the bucket public to fix AccessDenied.

Recipe 08

Serve Private S3 Content Through CloudFront

Solve secure content delivery using S3 + CloudFront rather than exposing the bucket directly. Include HTTPS, origin access, caching, invalidation, and the request path from user to object.

- ✓ SECURE
- ✓ FAST
- ✓ SCALABLE
- ✓ GLOBAL
- ✓ PRODUCTION READY

LEARN
BUILD
DEPLOY
GROW

THE PROBLEM

- You want to serve files (images, videos, downloads, static web content) from S3, but you don't want the bucket to be public.
- You need a secure, fast and global solution.
- You want to use HTTPS, caching and control who can access the content.
- You want to avoid exposing your S3 bucket directly to the internet.

THE SOLUTION

- Keep the S3 bucket private (block all public access).
- Use CloudFront as the content delivery layer.
- Configure an Origin Access Control (OAC) for secure access from CloudFront to S3.
- Use HTTPS with a custom domain (optional).
- Enable caching and set appropriate cache behaviors.
- Use invalidation when content changes.

KEY BENEFITS

- ✓ S3 bucket stays private (no public access).
- ✓ Global content delivery (faster for users).
- ✓ HTTPS everywhere.
- ✓ Reduced load and cost on S3.
- ✓ Fine-grained access control.
- ✓ Caching improves performance.
- ✓ Easy content invalidation.
- ✓ Industry best practice.

ARCHITECTURE DIAGRAM



Users request content from CloudFront. CloudFront fetches the object from the private S3 bucket using Origin Access Control, caches it, and serves it over HTTPS.

Key Components

- CloudFront Distribution
- S3 Bucket (Private)
- Origin Access Control (OAC)
- SSL/TLS Certificate (ACM)
- Cache Behavior
- Custom Domain (Route 53)
- Invalidation
- Bucket Policy
- Optional: WAF (security)

BUCKET POLICY (EXAMPLE)

Allow CloudFront (OAC) to access your S3 bucket

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "cloudfront.amazonaws.com"
      },
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3::your-bucket-name/*",
      "Condition": {
        "StringEquals": {
          "AWS:SourceArn": "arn:aws:cloudfront::123456789012:distribution/EXAMPLE"
        }
      }
    }
  ]
}
```

CACHING SETTINGS (EXAMPLE)

Cache Policy	Use managed policy (CachingOptimized) or custom.
TTL (Default)	e.g. 1 day (86400 seconds)
Minimum TTL	e.g. 1 hour (3600 seconds)
Maximum TTL	e.g. 1 year (31536000 seconds)
Compress Objects	Yes (for text files)
Cache Based on	URL, headers, cookies (as needed)

INVALIDATION

- Use invalidation when you update or delete content.
- Invalidate specific files or paths.
- ✓ Example (AWS CLI):

```
aws cloudfront create-invalidation \
  --distribution-id E123ABC456 \
  --paths "/images/logo.png"
```

- Or invalidate everything:

```
aws cloudfront create-invalidation \
  --distribution-id E123ABC456 \
  --paths "/"
```

TROUBLESHOOTING

Issue	Possible Cause / Fix
403 AccessDenied	Check bucket policy, OAC, and object permissions.
404 Not Found	Check object key (path) and file name.
Content not updating	Invalidate the cache or wait for TTL to expire.
SSL certificate error	Check ACM certificate and CloudFront domain.
Slow performance	Check cache settings and edge locations.

BEST PRACTICES

- Keep S3 bucket private (block all public access).
- Use Origin Access Control (OAC) instead of OAI (recommended).
- Use HTTPS with a custom domain (ACM).
- Set appropriate cache policies.
- Use invalidation for content changes.
- Use versioned file names for static assets.
- Consider using AWS WAF for added security.
- Monitor with CloudWatch and CloudFront logs.

GLOBAL
DELIVERY
REAL IMPACT

Recipe 09

Stop Paying Forever for Old S3 Data

Create lifecycle rules that transition or expire objects according to business needs. Show a practical logs/backups example, storage classes, retention decisions, and the danger of deleting data too aggressively.

- ✓ LIFECYCLE RULES
- ✓ COST SAVINGS
- ✓ RIGHT STORAGE CLASS
- ✓ RETENTION PLAN
- ✓ AVOID DATA LOSS

LEARN
BUILD
DEPLOY
GROW

THE PROBLEM

- Old data stays in S3 forever, increasing storage costs.
- Logs, backups and temporary files accumulate over time.
- Keeping everything in Standard storage is expensive.
- No lifecycle rules = wasted money and manual cleanup.
- Deleting data too aggressively can cause data loss and compliance issues.

THE SOLUTION

- Use S3 Lifecycle Rules to transition or expire objects automatically.
- Apply the right storage class for each data type (logs, backups, archives).
- Set retention periods based on business and compliance needs.
- Use prefixes or tags to target specific data.
- Always test and monitor lifecycle rules before full deployment.

KEY BENEFITS

- ✓ Lower storage costs.
- ✓ Better data organization.
- ✓ Automatic lifecycle management.
- ✓ Meet compliance and retention requirements.
- ✓ Improved operational efficiency.
- ✓ Keep important data safe and accessible.

PRACTICAL EXAMPLE – LOGS & BACKUPS



Example: Keep logs in Standard for 7 days, move to IA for 30 days, then Glacier for 1 year, and delete after 2 years.

COMMON STORAGE CLASSES

Storage Class	Use Case	Cost
Standard	Frequent access (near real-time)	Highest
Standard-IA	Infrequent access	Lower
One Zone-IA	Infrequent, non-critical	Lower
Glacier Instant Retrieval	Archive, occasional	Low
Deep Archive	Long-term archive	Lowest

LIFECYCLE RULES EXAMPLE (S3 CONSOLE/JSON)

```
{
  "Rules": [
    {
      "ID": "logs-lifecycle",
      "Status": "Enabled",
      "Prefix": "logs/",
      "Transitions": [
        {
          "Days": 7, "StorageClass": "STANDARD_IA"
        },
        {
          "Days": 30, "StorageClass": "GLACIER"
        }
      ],
      "Expiration": { "Days": 730 }
    }
  ]
}
```

RETENTION DECISIONS

Data Type	Keep For	Why
Application logs	7–30 days	Troubleshooting & monitoring
Backups	30–90 days	Recovery needs
Compliance data	1–7 years	Regulatory requirements
Archives	Long term	Historical reference
Temp files	1–7 days	Not needed

DANGER: DON'T DELETE TOO AGGRESSIVELY

- You may lose important data (such as logs, backups or audits).
- It can break your application (if it relies on old files).
- It may cause compliance violations.
- Recovery can be impossible (depending on the data type).
- Always confirm retention needs before setting expiration rules.

TROUBLESHOOTING – IF DATA IS STILL NOT TRANSITIONING

Check	Fix
Lifecycle rule not enabled	Enable and save the rule.
Wrong prefix or tag	Verify prefix/tag matches your objects.
Objects are in the wrong class	List objects and check storage class.
Rules not applied yet	Wait for the next evaluation cycle (up to 24 hours).
Bucket versioning	Check if versions are being retained.

USEFUL CLI COMMANDS

```
# List lifecycle configuration
aws s3api get-bucket-lifecycle-configuration \
  --bucket my-bucket

# Check objects and storage class
aws s3api list-objects-v2 \
  --bucket my-bucket --query 'Contents[*].[Key,StorageClass]' \
  --output table

# Manually transition (if needed)
aws s3api put-object-retention ...
```

BEST PRACTICES

- ✓ Enable versioning for important data.
- ✓ Use lifecycle rules for all buckets.
- ✓ Apply the right storage class.
- ✓ Set retention based on business needs.
- ✓ Monitor costs with AWS Cost Explorer.
- ✓ Test rules in a non-production bucket.
- ✓ Document your retention policy.

SMART STORAGE
SAVES MONEY

Recipe 10

Recover an EC2 Server After Someone Breaks It

- ✓ BACKUP
- ✓ RECOVER
- ✓ BE PREPARED
- ✓ PROVE IT WORKS

LEARN
BUILD
DEPLOY
GROW

Use EBS Snapshots and AWS Backup to recover data or rebuild the workload. Learn the snapshot → restore → attach/recover → verify process, and understand the difference between backing something up and proving it can be restored.

THE PROBLEM

- An EC2 server is broken (accidental deletion, bad configuration, failed update, malware, etc).
- You need to recover the server and its data quickly.
- You don't want to lose important data.
- Backups exist, but no one has ever tested the recovery process.
- You need a reliable, repeatable recovery method.

THE SOLUTION

- Use EBS snapshots and/or AWS Backup to recover.
- Restore the instance from a snapshot or attach the snapshot to a new volume.
- Rebuild the EC2 instance if needed (using the snapshot).
- Attach and recover the data.
- Verify the application and data are working.
- Regularly test restores, not just create backups.

KEY BENEFITS

- ✓ Recover from human error, malware, or system failure.
- ✓ Minimize downtime.
- ✓ Protect important data.
- ✓ Flexible recovery options (same instance or new instance).
- ✓ Supports disaster recovery and business continuity.
- ✓ Builds confidence that backups actually work.
- ✓ Follows AWS best practices.

ARCHITECTURE & RECOVERY FLOW



RECOVERY OPTIONS

- 1 Launch a new EC2 instance from the snapshot (new volume).
- 2 Attach the snapshot as a new EBS volume to an existing instance.
- 3 Replace a failed instance with a new instance and attach restored volumes.
- 4 Use AWS Backup for centralized, automated backup and recovery.

AWS CLI EXAMPLES

Create a snapshot

```
aws ec2 create-snapshot \
  --volume-id vol-0123456789abcdef0 \
  --description "Backup before change"
```

List snapshots

```
aws ec2 describe-snapshots \
  --owner-ids self
```

Create a volume from snapshot

```
aws ec2 create-volume \
  --snapshot-id snap-0123456789abcdef0 \
  --availability-zone us-east-1a
```

RECOVERY STEPS (DETAILED)

- 1 Identify the problem
Confirm what is broken and what needs to be recovered.
- 2 Find the right snapshot
Use time, tags, or AWS Backup to locate the correct snapshot.
- 3 Restore the volume or instance
Create a new volume from the snapshot or launch a new EC2 instance using the snapshot.
- 4 Attach and recover
Attach the volume or use it as the root volume. Mount the volume if it's a data volume.
- 5 Verify
Check the application, data, and logs to confirm everything is working as expected.
- 6 Update and improve
Document the recovery, fix the root cause, and improve backup policies if needed.

BACKUP vs RECOVERY

- | Backup | Recovery |
|---|--|
| ● Creating snapshots or using AWS Backup. | ● Restoring from snapshots or AWS Backup. |
| ● Storing data safely. | ● Rebuilding the workload. |
| ● Having a copy. | ● Verifying the application and data work. |
| ● Does NOT guarantee you can restore it. | ● Proves your backups actually work. |

"A backup is only useful if you can restore it."
Test your recovery process regularly.

COMMON ISSUES & FIXES

Issue	Fix
Snapshot not found	Check region, account and tags.
Volume won't attach	Verify instance state, AZ and volume type.
Instance still not working	Check configuration, user data and logs.
Data is missing	Ensure you used the correct snapshot.
Application errors	Check dependencies and reconfigure if needed.

BEST PRACTICES

- Use AWS Backup for automated, centralized backups.
- Tag snapshots with purpose and retention period.
- Keep snapshots in multiple regions for critical data.
- Regularly test the full restore process.
- Use lifecycle policies to control snapshot costs.
- Monitor backup and recovery status.
- Document your recovery runbook.

USEFUL AWS CLI COMMANDS

```
# List backups (AWS Backup)
aws backup list-backup-jobs

# Restore an EC2 instance (via console or create-image)
aws ec2 create-image --instance-id i-0123456789abcdef0 \
  --name "recovery-image"

# List volumes
aws ec2 describe-volumes

# Attach a volume
aws ec2 attach-volume --volume-id vol-abc123 --instance-id i-abc123 \
  --device /dev/svdf
```

Recipe 11

Connect EC2 to RDS Without Exposing the Database Publicly

Build the common application → private RDS architecture. Configure security-group-to-security-group access, subnets, DNS endpoint, port, credentials, and diagnose connection timeouts versus authentication failures.

✓ PRIVATE
✓ SECURE
✓ BEST PRACTICE
✓ PRODUCTION READY

LEARN
BUILD
DEPLOY
GROW

THE PROBLEM

- You want your EC2 application to connect to RDS.
- You do not want the database to be publicly accessible.
- You need a secure, reliable connection inside your VPC.
- Connection can fail due to network, security group, DNS, port or authentication issues.
- You want a clear troubleshooting process.

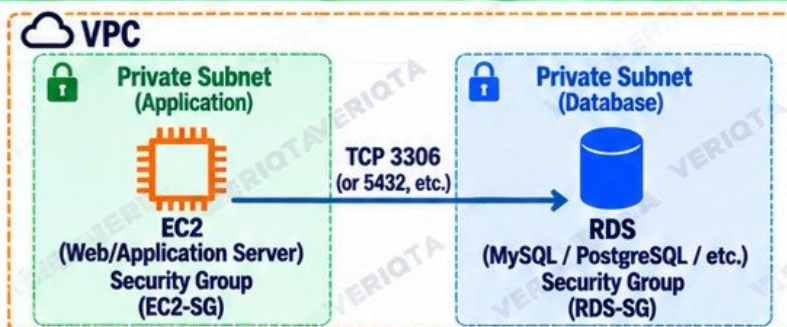
THE SOLUTION

- Place EC2 and RDS in the same VPC (in private subnets).
- Keep RDS public access disabled.
- Use security-group-to-security-group rules (allow EC2 SG to access RDS SG on the database port).
- Use the RDS DNS endpoint.
- Use the correct port and database credentials.
- Test connectivity and diagnose issues properly.

KEY BENEFITS

- ✓ Database is not exposed to the internet.
- ✓ More secure architecture.
- ✓ Uses AWS best practices.
- ✓ Networks are isolated in private subnets.
- ✓ Granular access control with security groups.
- ✓ Reliable and scalable.
- ✓ Suitable for production workloads.

ARCHITECTURE DIAGRAM



EC2 connects to RDS using the RDS DNS endpoint over a private network (no internet).

SECURITY GROUP CONFIGURATION

EC2 Security Group (EC2-SG)

- Outbound: Allow TCP to RDS-SG on database port (e.g. 3306 for MySQL, 5432 for PostgreSQL)
- Inbound: As required for your application (e.g. HTTP/HTTPS, SSH from bastion or SSM)

RDS Security Group (RDS-SG)

- Inbound: Allow TCP from EC2-SG on database port (e.g. 3306 or 5432)
- Outbound: Usually not required (or keep default)

- ✓ Use security-group-to-security-group rules instead of IP addresses for better security and easier management.

CONNECTION DETAILS

RDS Endpoint	mydb.abc123xyz.us-east-1.rds.amazonaws.com
Port (MySQL)	3306
Port (PostgreSQL)	5432
Database Name	ecommerce
Username	appuser
Password	Use AWS Secrets Manager (or environment variable)
Network	Private subnets (no public access)
RDS Public Access	Disabled

TEST CONNECTION (EXAMPLES)

- Test DNS resolution
`nslookup mydb.abc123xyz.us-east-1.rds.amazonaws.com`
- Test port connectivity (from EC2)
`nc -zv mydb.abc123xyz.us-east-1.rds.amazonaws.com 3306`
For PostgreSQL
`nc -zv mydb.abc123xyz.us-east-1.rds.amazonaws.com 5432`
- Test with database client (example MySQL)
`mysql -h mydb.abc123xyz.us-east-1.rds.amazonaws.com \\\n-P 3306 -u appuser -p`
- Test with database client (example PostgreSQL)
`psql -h mydb.abc123xyz.us-east-1.rds.amazonaws.com \\\n-p 5432 -U appuser -d ecommerce`

TROUBLESHOOTING

Issue	Symptoms	How to Fix
Connection Timeout	Cannot connect, request times out	Check security groups, NACLs, routes, subnets, port, same VPC/peering.
Authentication Failure	Access denied for user	Check username, password, database name, IAM auth (if used).
DNS Resolution Failure	Unknown host or no address	Check VPC DNS settings (enable DNS resolution and DNS hostnames).
Wrong Port	Connection refused	Use correct port (3306 for MySQL, 5432 for PostgreSQL).
RDS Not Accessible	Still cannot connect	Verify RDS is in private subnets and public access is disabled.

BEST PRACTICES

- ✓ Keep RDS in private subnets.
- ✓ Use security-group-to-security-group rules.
- ✓ Use AWS Secrets Manager for credentials.
- ✓ Enable encryption at rest and in transit (SSL/TLS).
- ✓ Use IAM database authentication where possible.
- ✓ Monitor with CloudWatch and enable RDS logs.
- ✓ Test connectivity after any network or security change.

COMMON MISTAKES

- ✗ Making RDS publicly accessible.
- ✗ Using 0.0.0.0/0 in security group rules.
- ✗ Using the wrong port.
- ✗ Using the wrong database endpoint.
- ✗ Hardcoding credentials in code.
- ✗ Forgetting to enable VPC DNS resolution.
- ✗ Assuming a timeout is always a credential problem.

QUICK CHECKLIST

- ✓ RDS is in private subnets.
- ✓ RDS public access is disabled.
- ✓ Security group rules are correct.
- ✓ EC2 can resolve RDS endpoint (DNS).
- ✓ Port is open from EC2 to RDS.
- ✓ Credentials are correct.
- ✓ Connection is tested and works.

Recipe 12

Fix an Application That Is Exhausting RDS Connections

Treat this as an operational incident. Observe connection count, identify connection storms or poor pooling, introduce RDS Proxy where appropriate, protect credentials, and verify recovery.

- ✓ STABLE DATABASES
- ✓ BETTER PERFORMANCE
- ✓ HIGHER RELIABILITY
- ✓ PRODUCTION READY

LEARN
BUILD
DEPLOY
GROW

THE PROBLEM

- Application opens too many connections to the database.
- You see errors like "too many connections" or "connection refused".
- Database becomes slow or unavailable.
- Can be caused by missing connection pooling, short-lived connections, multiple app instances, or traffic spikes.
- Leads to downtime, failed requests and poor user experience.

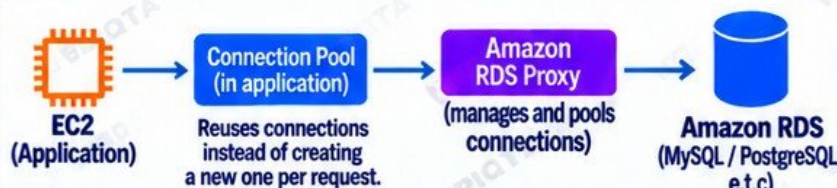
THE SOLUTION

- Identify the root cause (monitor, logs, application behaviour).
- Use connection pooling in the application.
- Introduce Amazon RDS Proxy (where appropriate) to manage and pool connections.
- Protect credentials using AWS Secrets Manager (no hardcoded secrets).
- Optimize database parameters (max_connections).
- Verify recovery and monitor to ensure it does not happen again.

KEY BENEFITS

- ✓ Prevents connection exhaustion.
- ✓ Improves application stability and performance.
- ✓ Handles traffic spikes more easily.
- ✓ Reduces database load.
- ✓ Centralized connection management with RDS Proxy.
- ✓ More secure credential management.
- ✓ Better operational visibility.
- ✓ Industry best practice for production environments.

ARCHITECTURE DIAGRAM



The application connects to RDS Proxy instead of connecting directly to the database. RDS Proxy manages and reuses database connections, reducing the number of connections to RDS.

MONITORING (CLOUDWATCH)

- Monitor DatabaseConnections.
- Check MaxConnections and current usage.
- Look for sudden spikes (connection storms).
- Set CloudWatch alarms (e.g. > 80% of max).
- Review database metrics, logs and slow queries.
- Use Enhanced Monitoring for deeper visibility.

COMMON ERROR MESSAGES

- "too many connections"
- "connection refused"
- "could not connect to server"
- "remaining connection slots are reserved"
- "timeout"

CHECK CURRENT CONNECTIONS

MySQL (on RDS)

```
SHOW STATUS LIKE 'Threads_connected';
SHOW VARIABLES LIKE 'max_connections';
```

PostgreSQL (on RDS)

```
SELECT count(*) FROM pg_stat_activity;
SHOW max_connections;
```

AWS CLI (CloudWatch metric)

```
aws cloudwatch get-metric-statistics \
  --namespace AWS/RDS \
  --metric-name DatabaseConnections \
  --dimensions Name=DBInstanceIdentifier,Value=mydb \
  --statistics Average --period 300 --region us-east-1
```

USE RDS PROXY (WHEN NEEDED)

- 1 Create an RDS Proxy in the same VPC.
- 2 Configure the target RDS database.
- 3 Update the application to use the RDS Proxy endpoint.
- 4 Use IAM authentication or AWS Secrets Manager for credentials.
- 5 Test under load to confirm connections are managed properly.

RDS Proxy is especially useful for serverless applications, microservices and applications with many short-lived connections.

TROUBLESHOOTING STEPS

- 1 Check current connection count and limits.
- 2 Identify if traffic spike or connection leak.
- 3 Review application code for missing pooling.
- 4 Check if multiple app instances are creating new connections.
- 5 Verify database parameters and logs.
- 6 Consider using RDS Proxy.
- 7 Test after applying fixes.
- 8 Set up monitoring and alarms to prevent recurrence.

BEST PRACTICES

- Always use connection pooling in the application.
- Use RDS Proxy for high-scale or serverless workloads.
- Store credentials in AWS Secrets Manager.
- Set appropriate max_connections value.
- Monitor connection count and set alarms.
- Load test before production.
- Review and optimize regularly.

COMMON CAUSES

- No connection pooling in the application.
- Application creates a new connection per request.
- Traffic spike (e.g. deployment or event).
- Too many application instances.
- Idle connections not closed.
- Long-running queries holding connections.
- Connection leaks in the code.
- Incorrect database configuration.

KEY TAKEAWAY

“It's not enough to back the database. You must design the application to use connections efficiently, monitor continuously and put safeguards in place like RDS Proxy. A stable database keeps your business running.”

Recipe 13

Store and Rotate Database Credentials Without Putting Secrets in Code

- ✓ NO SECRETS IN CODE
- ✓ USE SECRETS MANAGER
- ✓ AUTOMATE ROTATION
- ✓ SECURE BY DESIGN

LEARN
BUILD
DEPLOY
GROW

Use AWS Secrets Manager and IAM permissions. Show application → IAM role → Secrets Manager → database. Include rotation thinking and troubleshoot permissions without exposing the secret.

THE PROBLEM

- Credentials hardcoded in code, config files or environment variables.
- Secrets exposed in Git, logs or server files.
- Manual password management and rotation.
- Difficulty rotating credentials without downtime.
- Security and compliance risks.

THE SOLUTION

- Use AWS Secrets Manager to store database credentials.
- Give the application an IAM role with permission to read the secret.
- Application retrieves the secret at runtime (instead of hardcoding).
- Enable automatic rotation (where supported).
- Use least privilege IAM permissions.
- Monitor access and rotate regularly.

KEY BENEFITS

- ✓ No secrets in code.
- ✓ Automatic and safe rotation.
- ✓ Improved security and compliance.
- ✓ Centralized secret management.
- ✓ Least privilege access with IAM.
- ✓ Easy to audit and monitor.
- ✓ Reduces risk of credential leaks.
- ✓ Works with many AWS and third-party databases.

ARCHITECTURE DIAGRAM



The application uses its IAM role to retrieve the database credentials from Secrets Manager at runtime. The secret is never stored in code.

ROTATION CONCEPT

- 1 Secrets Manager can automatically rotate database credentials (e.g. RDS, Aurora, etc.).
- 2 Rotation creates a new password, updates the database, and stores the new secret version.
- 3 The application continues to use the secret (usually no code change needed).
- 4 You can test rotation manually and monitor the process.

EXAMPLE: RETRIEVE SECRET (AWS CLI)

```
# Get secret value (returns encrypted JSON)
aws secretsmanager get-secret-value \
  --secret-id my-db-secret \
  --query SecretString --output text

# Example output (do not log this in real applications)
{
  "username": "dbuser",
  "password": "*****",
  "host": "mydb.abc123xyz.us-east-1.rds.amazonaws.com",
  "port": 3306,
  "dbname": "appdb"
}
```

EXAMPLE: IAM POLICY (LEAST PRIVILEGE)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "secretsmanager:GetSecretValue"
      ],
      "Resource": "arn:aws:secretsmanager:us-east-1:123456789012:secret:my-db-secret"
    }
  ]
}
```

TROUBLESHOOTING

Issue	Possible Cause	How to Fix
AccessDenied error	IAM role does not have permission	Add secretsmanager: GetSecretValue permission
Secret not found	Wrong secret ID or region	Check secret name and region
Cannot connect to DB	Wrong credentials or rotation issue	Verify secret value and test manually
Rotation failed	Missing permissions for rotation	Check rotation Lambda role and logs
Application still using old password	Caching the secret in application	Reduce cache time or refresh secret on retry

BEST PRACTICES

- Never hardcode secrets in code, configs or logs.
- Use IAM roles instead of access keys.
- Enable automatic rotation where possible.
- Grant least privilege permissions.
- Monitor access with CloudTrail.
- Do not log secret values.
- Set a reasonable cache/refresh strategy.

COMMON MISTAKES

- Storing credentials in environment variables or configuration files.
- Giving overly broad IAM permissions.
- Logging secret values.
- Not testing the rotation process.
- Hardcoding the secret ARN in multiple places.
- Ignoring error handling when retrieving secrets.
- Caching secrets for too long.

USEFUL AWS CLI COMMANDS

```
# Describe secret
aws secretsmanager describe-secret \
  --secret-id my-db-secret

# Get secret value
aws secretsmanager get-secret-value \
  --secret-id my-db-secret

# Rotate secret manually
aws secretsmanager rotate-secret \
  --secret-id my-db-secret

# Check rotation status
aws secretsmanager describe-rotation-status \
  --secret-id my-db-secret
```

Recipe 14

Trigger Work Automatically When an Event Happens

- ✓ EVENT-DRIVEN
- ✓ AUTOMATE TASKS
- ✓ SCALABLE
- ✓ REAL-WORLD READY

LEARN
BUILD
DEPLOY
GROW

Build an event-driven architecture: S3 upload → EventBridge (or S3 event notification) → Lambda → processing. Learn about retries, permissions, logs, duplicate events, and how to troubleshoot when the function never runs.

THE PROBLEM

- You want work to happen automatically when a file is uploaded to S3 (e.g., process images, parse data, generate reports).
- Manual processing is slow, error-prone and does not scale.
- You need a reliable, serverless, event-driven solution.
- You also need to handle retries, permissions, logs and duplicate events.

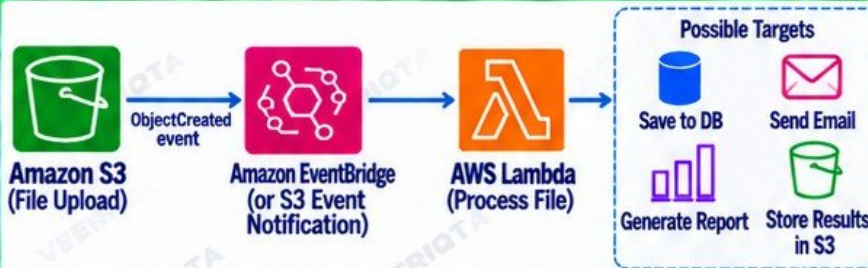
THE SOLUTION

- Use S3 event notifications or Amazon EventBridge.
- Trigger an AWS Lambda function to process the event.
- Set the correct IAM permissions.
- Implement error handling and retries.
- Use CloudWatch Logs to monitor execution.
- Be aware of duplicate events.
- Test and validate the end-to-end flow.

KEY BENEFITS

- ✓ Automatic and real-time processing.
- ✓ No servers to manage.
- ✓ Scales with demand.
- ✓ Reliable with built-in retries.
- ✓ Integrates with many AWS services.
- ✓ Cost-effective (pay only for usage).
- ✓ Improved operational efficiency.
- ✓ Common pattern used in real production systems.

ARCHITECTURE DIAGRAM



Upload a file to S3 → EventBridge (or S3 notification) detects the event → Lambda runs → processes the file → stores the result or takes further action.

EVENT FLOW STEPS

- 1 A file is uploaded to an S3 bucket.
- 2 S3 sends an event to EventBridge (or directly triggers Lambda).
- 3 EventBridge matches the event rule (if used) and invokes the Lambda function.
- 4 Lambda processes the file (e.g., resize image, parse data, store in DB).
- 5 Lambda logs the execution in CloudWatch Logs.
- 6 If the function fails, AWS retries automatically (with backoff).

LAMBDA FUNCTION EXAMPLE (PYTHON)

```

import json
import boto3

def lambda_handler(event, context):
    print("Event received:", json.dumps(event))
    for record in event['Records']:
        bucket = record['s3']['bucket']['name']
        key = record['s3']['object']['key']
        print(f"Processing file: s3://{bucket}/{key}")
        # TODO: Add your processing logic here
    return {
        'statusCode': 200,
        'body': 'File processed successfully'
    }
  
```

IAM PERMISSIONS (KEY POINTS)

- Lambda Execution Role needs:
- CloudWatch Logs permissions (AWSLambdaBasicExecutionRole)
 - S3 read permissions (if reading the object)
 - Permissions to write to other AWS services (e.g., S3, DynamoDB, SNS, etc.)
- Example policy (S3 read only):
- ```

{
 "Effect": "Allow",
 "Action": ["s3:GetObject"],
 "Resource": "arn:aws:s3:::my-bucket/*"
}

```

## RETRIES AND ERROR HANDLING

- Lambda automatically retries failed invocations (up to 2 retries for async).
- Use a Dead Letter Queue (DLQ) or on-failure destination (e.g., SQS) for failed events.
- Implement idempotent code (handle duplicate events).
- Set appropriate timeout and memory.
- Log errors and track failed events.

## TROUBLESHOOTING: FUNCTION NEVER RUNS

- Check if S3 event notification or EventBridge rule is correctly configured.
- Verify IAM permissions (S3, EventBridge, Lambda).
- Check CloudTrail for failed invocations.
- Look at CloudWatch Logs (is the function being called?).
- Confirm the event pattern (if using EventBridge).
- Check if the file type or prefix/suffix filters are correct.
- Test with a simple event (e.g., upload a test file).

## COMMON MISTAKES

- Missing IAM permissions.
- Incorrect event pattern or bucket configuration.
- Assuming events are never duplicated.
- Not handling errors or retries.
- Hardcoding values instead of using environment variables.
- Function timeout too short.
- Not checking CloudWatch Logs.
- Forgetting to test the full flow.

## BEST PRACTICES

- Use least privilege IAM permissions.
- Make your function idempotent.
- Use environment variables for configuration.
- Enable logging and monitoring.
- Set up DLQ or on-failure destination.
- Filter events (prefix/suffix) to reduce noise.
- Test thoroughly in a non-production environment.
- Monitor metrics and set CloudWatch alarms.

## Recipe 15

# Build, Scan, and Store a Container Image in ECR

✓ BUILD  
✓ SCAN  
✓ STORE  
✓ DEPLOY READY

LEARN  
BUILD  
DEPLOY  
GROW

Take an application image from local/CI build → authentication → ECR → vulnerability scanning → deployment-ready artifact. Include tag strategy, failed authentication, and image-pull problems.



## THE PROBLEM

- You need to store container images securely and manage versions.
- You want to scan images for vulnerabilities before deployment.
- Authentication, permission or network issues can prevent push or pull.
- Unscanned images may contain known vulnerabilities.
- Incorrect tags or using latest only makes rollbacks difficult.



## THE SOLUTION

- Build the container image (locally or in CI).
- Authenticate to ECR using AWS CLI or IAM role.
- Push the image to ECR.
- Enable ECR image scanning (Inspector).
- Use a clear tag strategy (versioned tags).
- Review scan results and fix critical vulnerabilities.
- Pull the image from ECR for deployment (ECS, EKS, EC2, etc.).



## KEY BENEFITS

- ✓ Secure and private container registry.
- ✓ Automatic vulnerability scanning (with Amazon Inspector).
- ✓ Integrates with CI/CD pipelines.
- ✓ Fine-grained access control using IAM.
- ✓ Reliable image storage and versioning.
- ✓ Supports rollback and reproducibility.
- ✓ Detects security issues early.
- ✓ Production-ready container images.



## ARCHITECTURE DIAGRAM



Build once, scan for vulnerabilities, store in ECR, and use the same image for any environment (dev, test, prod).



## AWS CLI COMMANDS

- 1 Authenticate to ECR  

```
aws ecr get-login-password --region us-east-1 | \
 docker login --username AWS \
 --password-stdin 123456789012.dkr.ecr.us-east-1.amazonaws.com
```
- 2 Build the image  

```
docker build -t myapp:1.0 .
```
- 3 Tag the image  

```
docker tag myapp:1.0 123456789012.dkr.ecr.us-east-1.amazonaws.com/myapp:1.0
```
- 4 Push the image  

```
docker push 123456789012.dkr.ecr.us-east-1.amazonaws.com/myapp:1.0
```
- 5 View images in ECR  

```
aws ecr describe-images --repository-name myapp
```



## TAG STRATEGY

- Use meaningful, versioned tags.
- Examples:
  - myapp:1.0.0
  - myapp:1.0.1
  - myapp:prod
  - myapp:staging
  - myapp:2024-09-10
  - Git SHA: myapp:abc1234
- Avoid using latest only.
- Use semantic versioning or date-based tags.
- Keep tags consistent across environments.



## VULNERABILITY SCANNING

- ECR uses Amazon Inspector for image scanning.
- Scanning can be set to scan on push or continuous scanning.
- Review findings in the ECR console.
- Fix critical and high-severity vulnerabilities.
- Integrate scanning into your CI/CD pipeline (fail the build if critical vulnerabilities are found).

A clean scan does not remove the need for runtime security and other controls.



## COMMON ISSUES & FIXES

| Issue                      | Cause                                            | Fix                                                                   |
|----------------------------|--------------------------------------------------|-----------------------------------------------------------------------|
| Authentication failed      | Invalid AWS credentials or expired token         | Run get-login-password again or check IAM permissions                 |
| Access denied              | Missing IAM permissions (ecr:PutImage)           | Attach correct IAM policy (e.g. AmazonEC2ContainerRegistryFullAccess) |
| Repository not found       | Wrong repository name or region                  | Check repository name and AWS region                                  |
| Image pull fails           | Network, IAM or wrong tag                        | Check instance role, VPC endpoints, and image tag                     |
| Scan shows vulnerabilities | Using outdated base image or vulnerable packages | Update base image and dependencies, rebuild and push                  |



## BEST PRACTICES

- Use IAM roles (not long-lived keys) for EC2/EKS/EKS.
- Enable image scanning.
- Use versioned tags (no latest only).
- Keep base images and packages updated.
- Integrate with CI/CD pipelines.
- Set lifecycle policies to remove old images.
- Use ECR pull-through cache (for public images).
- Monitor scan findings and remediate quickly.
- Use private subnets with VPC endpoints for secure access.
- Implement image signing (e.g. with Sigstore or AWS Signer).



## USEFUL COMMANDS

```
Get authorization token
aws ecr get-login-password --region us-east-1

Describe repository
aws ecr describe-repositories

List images
aws ecr list-images --repository-name myapp

Describe image details
aws ecr describe-images --repository-name myapp

Delete an image
aws ecr batch-delete-image --repository-name myapp \
 --image-ids imageTag=1.0.0
```

## Recipe 16

# Run a Containerized Application Without Managing EC2 Servers

- ✓ NO EC2 SERVERS
- ✓ SERVERLESS CONTAINERS
- ✓ SCALABLE
- ✓ PRODUCTION READY

LEARN  
BUILD  
DEPLOY  
GROW

Deploy an application using ECS on Fargate with an ALB. Cover task definition, service, networking, IAM roles, health checks, logs, and the common "task starts and immediately stops" failure.

## THE PROBLEM

- You want to run a containerized application without managing EC2 servers.
- You need a scalable, highly available and cost-effective solution.
- You are unsure how to configure ECS Fargate, ALB, networking, IAM roles and health checks.
- Tasks start and immediately stop with errors.
- Application is not accessible or fails health checks.

## THE SOLUTION

- Use Amazon ECS with Fargate (serverless containers).
- Create a task definition for your container.
- Create a Virtual Private Cloud (VPC) with public and private subnets.
- Deploy a service with an Application Load Balancer (ALB).
- Configure IAM roles (task role and execution role).
- Set health checks and logging with CloudWatch.
- Follow best practices for security, scaling and monitoring.

## KEY BENEFITS

- ✓ No EC2 server management.
- ✓ Pay only for what you use.
- ✓ Automatic scaling.
- ✓ High availability with multiple subnets and AZs.
- ✓ Secure networking with private subnets.
- ✓ Integrated with ALB, CloudWatch and other AWS services.
- ✓ Production-ready and widely used in real-world environments.

## ARCHITECTURE DIAGRAM



Users access your application through an ALB. The ALB routes traffic to ECS Fargate tasks running in private subnets. Your container image is stored in ECR and logs/metrics go to CloudWatch.

## KEY ECS FARGATE COMPONENTS

- 1 Task Definition – defines container image, CPU, memory, port, environment variables and IAM roles.
- 2 Service – maintains the desired number of tasks and integrates with ALB.
- 3 Fargate – runs containers without managing EC2 servers.
- 4 Application Load Balancer (ALB) – distributes traffic and performs health checks.
- 5 VPC & Subnets – ALB in public subnets, tasks in private subnets.
- 6 IAM Roles – execution role (pull image, write logs), task role (access AWS services).
- 7 CloudWatch – view logs, metrics and alarms.

## TASK DEFINITION (EXAMPLE)

```

{
 "family": "myapp",
 "networkMode": "awsvpc",
 "requiresCompatibilities": ["FARGATE"],
 "cpu": "512",
 "memory": "1024",
 "containerDefinitions": [
 {
 "name": "myapp",
 "image": "123456789012.dkr.ecr.us-east-1.amazonaws.com/myapp:1.0.0",
 "portMappings": [{"containerPort": 80}],
 "environment": [{"name": "ENV", "value": "prod"}]
 }
]
}

```

## IAM ROLES

- Execution Role (used by ECS agent)**
- Pull container image from ECR
  - Write logs to CloudWatch
  - Use managed policy: AmazonECSTaskExecutionRolePolicy
- Task Role (used by your application)**
- Access AWS services (e.g. S3, RDS)
  - Grant least privilege permissions
  - Do not use the execution role for application access

## COMMON FAILURE: TASK STARTS AND IMMEDIATELY STOPS

- Check the container logs in CloudWatch.
- Verify the container is listening on the correct port.
- Ensure the health check path returns 200 OK.
- Check environment variables and secrets.
- Confirm the correct IAM roles and permissions.
- Verify the image exists in ECR and the tag is correct.
- Check for missing dependencies or configuration files.
- Look at the stopped reason in the ECS console.

## BEST PRACTICES

- Use Fargate to avoid server management.
- Keep tasks in private subnets.
- Use an ALB with health checks.
- Follow least privilege IAM permissions.
- Store secrets in AWS Secrets Manager.
- Enable logging to CloudWatch.
- Use versioned image tags (e.g. v1.0.0).
- Set up auto scaling based on CPU or custom metrics.

## COMMON MISTAKES

- Using latest tag in production.
- Incorrect port mapping or health check path.
- Missing IAM permissions.
- Tasks in public subnets.
- Hardcoding secrets in the image or environment.
- Not checking CloudWatch logs.
- Forgetting to update the service after a new image.
- No auto scaling (single task only).

## USEFUL AWS CLI COMMANDS

```

Create a cluster
aws ecs create-cluster --cluster-name myapp-cluster

Register a task definition
aws ecs register-task-definition --cli-input-json file://taskdef.json

Create a service
aws ecs create-service --cluster myapp-cluster \
 --service-name myapp-service \
 --task-definition myapp:1 \
 --desired-count 2 \
 --launch-type FARGATE \
 --network-configuration file://network-config.json

```

## Recipe 17

# Find Out Who Changed or Deleted an AWS Resource

- ✓ INVESTIGATE CHANGES
- ✓ IMPROVE SECURITY
- ✓ MEET COMPLIANCE
- ✓ BE PRODUCTION READY

LEARN  
BUILD  
DEPLOY  
GROW

Use AWS CloudTrail as an investigation recipe. Start with the incident question "Who changed this?" Then trace event history, identity, API action, source, timestamp, and affected resource.

## THE PROBLEM

- A resource was changed or deleted, and you need to know who did it.
- You need to find the exact API action, time, source IP and identity.
- You need to understand what was changed and from where.
- You need reliable audit logs for security, compliance and incident response.
- Without proper logging, you cannot investigate, attribute changes or prevent future incidents.

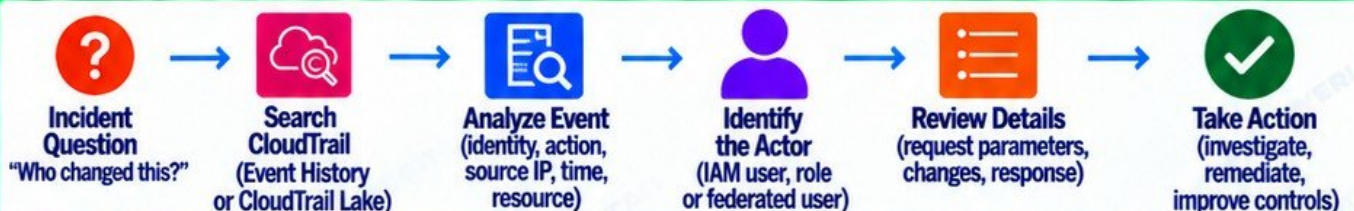
## THE SOLUTION

- Use AWS CloudTrail to view event history.
- Find the event for the specific resource and time window.
- Identify the IAM user, role or federated identity that made the change.
- Check the API action, source IP, user agent and request parameters.
- Review the affected resource and response.
- Use CloudTrail Event History for recent events and CloudTrail Lake for advanced searches.

## KEY BENEFITS

- ✓ Answer "who changed this" quickly.
- ✓ Full audit trail of API activity.
- ✓ Helps with security investigations and incident response.
- ✓ Supports compliance requirements (e.g., ISO, SOC 2, PCI DSS).
- ✓ Tracks changes across all AWS services.
- ✓ Deters unauthorized or accidental changes.
- ✓ Provides visibility into what, when, who and from where.

## INVESTIGATION FLOW



CloudTrail records API calls made in your AWS account. It shows who did what, when, from where and on which resource.

### EXAMPLE: CLOUDTRAIL EVENT (JSON)

```
{
 "eventTime": "2024-08-20T14:32:12Z",
 "eventName": "DeleteBucket",
 "eventSource": "s3.amazonaws.com",
 "userIdentity": {
 "type": "IAMUser",
 "userName": "john.doe",
 "arn": "arn:aws:iam::123456789012:user/john.doe"
 },
 "sourceIPAddress": "203.0.113.25",
 "userAgent": "aws-cli/2.15.0",
 "requestParameters": {
 "bucketName": "my-app-logs"
 },
 "responseElements": null,
 "eventID": "abcd1234-5678-90ef-ghij-1234567890ab"
}
```

### USEFUL AWS CLI COMMANDS

```
Lookup recent events for a resource
aws cloudtrail lookup-events \
 --lookup-attributes AttributeKey=ResourceName, \
 AttributeValue=my-app-logs \
 --max-results 50

Lookup events by event name (e.g. DeleteBucket)
aws cloudtrail lookup-events \
 --lookup-attributes AttributeKey=EventName, \
 AttributeValue=DeleteBucket

Lookup events by user identity
aws cloudtrail lookup-events \
 --lookup-attributes AttributeKey=Username, \
 AttributeValue=john.doe

Advanced query with CloudTrail Lake
aws cloudtrail start-query \
 --query-statement "SELECT eventTime, eventName, \
 userIdentity.userName, sourceIPAddress \
 FROM default WHERE resourceName = 'my-app-logs' \
 ORDER BY eventTime DESC"
```

### WHAT TO LOOK FOR

- eventTime – when it happened.
- eventName – the API action (e.g. ModifyInstance, DeleteBucket).
- userIdentity – who did it (IAM user, role, or federated user).
- sourceIPAddress – where the request came from.
- userAgent – tool used (AWS CLI, SDK, Console, etc.).
- requestParameters – what was changed (e.g. new configuration).
- responseElements – result of the action.
- resources – the affected resource ARN or name.

### COMMON SCENARIOS

- An EC2 instance was terminated.
- An S3 bucket was deleted.
- A security group rule was modified.
- An IAM policy was changed.
- A user's permissions were escalated.
- A resource was modified from an unexpected IP.

### TROUBLESHOOTING TIPS

- No events found? Check the correct region and time range.
- Ensure CloudTrail is enabled for all regions (organization trail recommended).
- Use CloudTrail Lake for advanced queries (more than 90 days).
- Check if the action was performed via AWS Console, CLI, SDK or third-party tools.
- Verify you have the required permissions to view logs.

### BEST PRACTICES

- Enable an organization trail (all regions).
- Send logs to a secure S3 bucket (with access controls).
- Enable CloudTrail log file integrity validation.
- Use CloudTrail Lake for long-term and advanced searches.
- Set up CloudWatch alarms for sensitive actions (e.g. DeleteBucket, TerminateInstances).
- Regularly review and monitor CloudTrail events.

## Recipe 18

# Alert Before an AWS Problem Becomes an Outage

Create useful CloudWatch monitoring rather than collecting metrics blindly.

Build metric → threshold → alarm → SNS notification. Use a realistic EC2, ALB, or RDS failure and discuss avoiding noisy alerts.

✓ MONITOR SMART  
✓ ALERT EARLY  
✓ REDUCE NOISE  
✓ KEEP SYSTEMS RELIABLE

LEARN  
BUILD  
DEPLOY  
GROW

## THE PROBLEM

- You only find out about issues when users complain.
- Important metrics are not monitored.
- Alerts are either missing or too noisy.
- You waste time reacting instead of preventing outages.
- Small issues (e.g., high CPU, failing health checks) are not detected early.

## THE SOLUTION

- Use Amazon CloudWatch to monitor key metrics.
- Set meaningful thresholds based on normal and abnormal behavior.
- Create CloudWatch alarms.
- Send notifications through Amazon SNS (email, SMS, Slack, etc.).
- Test your alarms.
- Tune thresholds to reduce noise.
- Use alarms for both infrastructure and application metrics.

## KEY BENEFITS

- ✓ Detect issues before they become outages.
- ✓ Faster response time.
- ✓ Improved reliability and uptime.
- ✓ Proactive instead of reactive operations.
- ✓ Less time spent troubleshooting.
- ✓ Better visibility into system health.
- ✓ Supports incident response and on-call workflows.

## MONITORING ARCHITECTURE



### EXAMPLE 1: EC2 HIGH CPU

- Metric: CPUUtilization
- Threshold: > 80% for 5 minutes
- Statistic: Average
- Period: 5 minutes
- Action: Send SNS notification
- Use case: Detect high CPU before the instance becomes unresponsive.

### EXAMPLE 2: ALB HIGH 5XX ERRORS

- Metric: HTTPCode\_Target\_5XX\_Count
- Threshold: > 50 for 5 minutes
- Statistic: Sum
- Period: 1 minute
- Action: Send SNS notification
- Use case: Detect application errors early and investigate before users are fully impacted.

### EXAMPLE 3: RDS LOW FREE STORAGE

- Metric: FreeStorageSpace
- Threshold: < 10 GB
- Statistic: Minimum
- Period: 5 minutes
- Action: Send SNS notification
- Use case: Prevent database from running out of storage.

## AVOIDING NOISY ALERTS

- Understand normal baseline behavior.
- Use realistic thresholds (not too sensitive).
- Use multiple data points (e.g., 3 out of 5).
- Filter out expected events (maintenance, deployments).
- Use composite alarms for related metrics.
- Review and tune alarms regularly.
- Alert on actionable issues, not everything.

## TROUBLESHOOTING TIPS

- Check if the metric is being published.
- Verify the threshold and comparison operator.
- Ensure the alarm state is correct (OK / ALARM / INSUFFICIENT\_DATA).
- Check SNS subscription and email spam folder.
- Look at CloudWatch metric graphs for patterns.
- Verify IAM permissions if using cross-account or custom metrics.

## USEFUL AWS CLI COMMANDS

- List metrics  
`aws cloudwatch list-metrics`
- Create an alarm  
`aws cloudwatch put-metric-alarm \\  
 --alarm-name MyAlarm ...`
- Describe alarms  
`aws cloudwatch describe-alarms`
- View alarm history  
`aws cloudwatch describe-alarm-history \\  
 --alarm-name MyAlarm`

## Recipe 19

# Find and Shut Down AWS Cost Leaks

Investigate common waste such as idle EC2 instances, unattached EBS volumes, forgotten snapshots, NAT Gateway traffic, oversized databases, and unnecessary data transfer. Finish with Budgets and cost controls so the problem does not quietly return.

✓ FIND WASTE  
✓ REDUCE COSTS  
✓ SET GUARDRAILS  
✓ KEEP IT EFFICIENT

LEARN  
BUILD  
DEPLOY  
GROW

## ⚠ THE PROBLEM

- AWS costs keep increasing.
- Idle resources are not cleaned up.
- Unattached EBS volumes and old snapshots accumulate.
- NAT Gateway and data transfer costs are higher than expected.
- Databases are overprovisioned.
- There is no budget or alert, so cost leaks keep coming back.

## ✅ THE SOLUTION

- Identify and remove unused resources.
- Right-size compute, storage and databases.
- Analyze NAT Gateway and data transfer costs.
- Use AWS Cost Explorer and AWS Trusted Advisor.
- Set AWS Budgets and alerts.
- Apply cost controls (tags, policies, instance schedules, auto-scaling).
- Review regularly to prevent future waste.

## 💡 KEY BENEFITS

- ✓ Lower AWS bills.
- ✓ Remove unused and idle resources.
- ✓ Better visibility into spending.
- ✓ Proactive alerts before costs grow.
- ✓ Improved resource utilization.
- ✓ More efficient and sustainable environment.
- ✓ Prevents cost leaks from returning.

## ⚙ COMMON AWS COST LEAKS



### Idle EC2 Instances

Stopped or low-usage instances still incurring costs (e.g., EBS).



### Unattached EBS Volumes

Volumes not attached to any instance but still charged.



### Forgotten Snapshots

Old snapshots accumulating over time.



### NAT Gateway Traffic

High data transfer costs through NAT Gateways.



### Oversized Databases

RDS/Aurora instances larger than needed (e.g., overprovisioned).



### Unnecessary Data Transfer

Cross-region or internet data transfer not required.

## 🔧 USEFUL AWS TOOLS

- AWS Cost Explorer – analyze spending.
- AWS Trusted Advisor – find cost optimizations.
- AWS Budgets – set cost and usage alerts.
- AWS Cost and Usage Reports (CUR) – detailed data.
- AWS Compute Optimizer – right-size resources.
- AWS Resource Groups – find unused resources.
- AWS Tagging – track ownership and environment.

## 🔍 EXAMPLE INVESTIGATION COMMANDS

- List stopped EC2 instances  
`aws ec2 describe-instances \\  
--filters "Name=instance-state-name,Values=stopped"`
- Find unattached EBS volumes  
`aws ec2 describe-volumes \\  
--filters "Name=status,Values=available"`
- List old snapshots  
`aws ec2 describe-snapshots \\  
--owner-ids self`
- Check NAT Gateway cost (Cost Explorer)  
`aws ce get-cost-and-usage \\  
--time-period Start=2024-01-01,End=2024-01-31 \\  
--metrics BlendedCost \\  
--group-by Type=DIMENSION,Key=UsageType`

## 📊 AWS BUDGETS AND COST CONTROLS

- Create AWS Budgets with email/SNS alerts.
- Set usage and cost thresholds.
- Use tags for cost allocation and ownership.
- Enable instance schedules (start/stop).
- Use auto-scaling instead of always-on instances.
- Right-size EC2, RDS and other resources.
- Use S3 lifecycle policies to delete old data.
- Review NAT Gateway usage and consider VPC endpoints.
- Implement IAM and SCP policies to prevent unapproved resources.

## ✅ BEST PRACTICES

- Review your costs regularly (weekly/monthly).
- Remove unused resources immediately.
- Use tags and naming standards.
- Right-size instead of overprovisioning.
- Use Savings Plans or Reserved Instances where appropriate.
- Set up Budgets and alerts from day one.
- Automate cleanup with scripts or Lambda.
- Make cost optimization a continuous process.

## ❌ COMMON MISTAKES

- Forgetting about stopped instances and EBS.
- Keeping old snapshots "just in case".
- Ignoring NAT Gateway and data transfer costs.
- Overprovisioning databases.
- Not using tags, making it hard to track costs.
- No budgets or alerts configured.
- Creating resources without limits or policies.
- Not reviewing costs regularly.

## 🎯 REAL-WORLD IMPACT

- Remove idle resources and save immediately.
- Reduce monthly AWS bills.
- Prevent unexpected cost spikes.
- Free up budget for new projects.
- Improve operational efficiency.
- Build a culture of cost awareness.
- Keep your environment lean, secure and sustainable.

## Recipe 20

# Build a Production-Ready Highly Available AWS Application

The final recipe combines the cookbook. A secure, scalable, highly available and production-ready architecture with all the supporting systems.

- ✓ HIGH AVAILABILITY
- ✓ SECURE BY DESIGN
- ✓ SCALABLE
- ✓ PRODUCTION READY

LEARN  
BUILD  
DEPLOY  
GROW

### ⚠ THE PROBLEM

- You need a production-ready application that is highly available, secure and scalable.
- You want to handle real-world traffic without downtime.
- You need to protect data and have reliable backups.
- You need centralized monitoring, logging and auditing.
- You want a design you can use and explain in interviews and real projects.

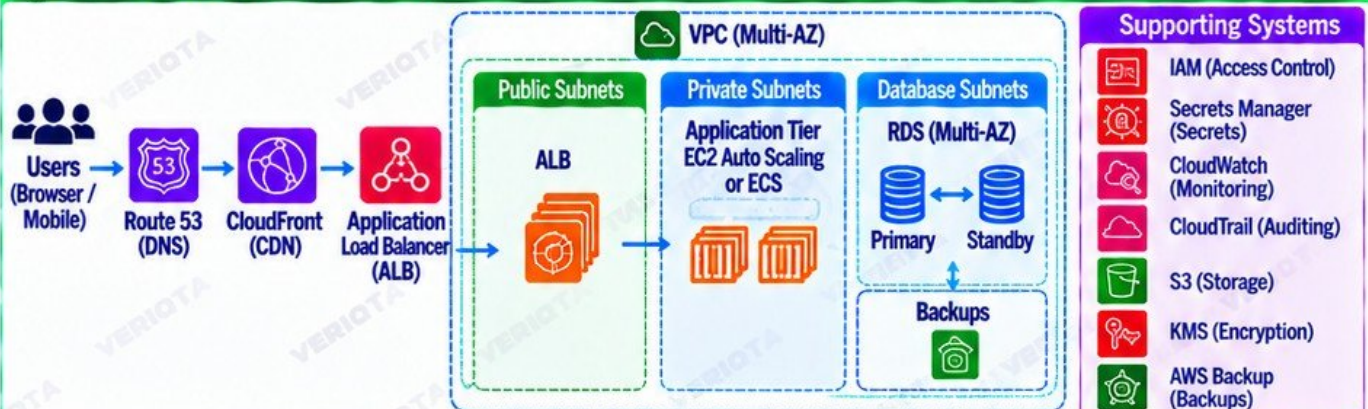
### ✓ THE SOLUTION

- Use a multi-layer AWS architecture with managed services.
- Distribute traffic using Route 53 and CloudFront.
- Use an Application Load Balancer and Auto Scaling (EC2 or ECS).
- Run the application in private subnets.
- Use RDS Multi-AZ for high availability.
- Enable backups and disaster recovery.
- Implement IAM, Secrets Manager, CloudWatch, CloudTrail, S3, KMS and AWS Backup.

### 💡 KEY BENEFITS

- ✓ High availability and fault tolerance.
- ✓ Better performance and global reach.
- ✓ Scalable and cost-optimized.
- ✓ Secure architecture with least privilege.
- ✓ Automatic failure recovery.
- ✓ Centralized monitoring, logging and auditing.
- ✓ Protected data with backups.
- ✓ Ready for production and real users.
- ✓ Uses AWS best practices.

### ⚙ ARCHITECTURE DIAGRAM



A highly available, secure and scalable AWS application with global reach, monitoring, auditing and backups.

### ⚙ KEY CONFIGURATIONS

- 1 Route 53 – DNS routing and health checks.
- 2 CloudFront – CDN for performance and DDoS protection.
- 3 ALB – distribute traffic across multiple targets.
- 4 EC2 Auto Scaling or ECS – run application in private subnets.
- 5 RDS Multi-AZ – automatic failover and high availability.
- 6 Security Groups – restrict traffic (least privilege).
- 7 IAM Roles – secure access for services and applications.

### ✓ BEST PRACTICES

- Use multiple Availability Zones.
- Keep application servers in private subnets.
- Use managed services where possible.
- Enable automatic scaling.
- Use least privilege IAM policies.
- Store secrets in Secrets Manager.
- Enable encryption (KMS).
- Set up monitoring, logging and alerting.
- Enable automated backups.
- Test failure scenarios regularly.

### ✗ COMMON MISTAKES

- Running databases in a single AZ.
- Exposing application servers to the internet.
- Weak or overly permissive security groups.
- Not enabling backups.
- Hardcoding credentials instead of Secrets Manager.
- No monitoring or alerts.
- Ignoring cost optimization.
- Not testing disaster recovery.
- Missing CloudTrail for auditing.
- Relying on one region.

### 🔧 USEFUL AWS CLI COMMANDS

```
Describe load balancers
aws elbv2 describe-load-balancers
Describe Auto Scaling groups
aws autoscaling describe-auto-scaling-groups
Check RDS instances
aws rds describe-db-instances
List S3 buckets (for backups)
aws s3 ls
View CloudTrail events
aws cloudtrail lookup-events --max-results 10
```

### 📊 REAL-WORLD OUTCOME

- Application stays online even if an instance fails.
- Automatically scales with traffic.
- Data is protected and can be recovered.
- You know what is happening (logs, metrics, alerts).
- You can audit who did what (CloudTrail).
- You can grow globally and keep costs under control.
- Ready for real users and production workloads.

### ➡ NEXT STEPS

- Implement this architecture in your lab.
- Practice failure scenarios (terminate instances, force failover).
- Set up dashboards and alarms.
- Test backup and restore.
- Optimize costs and review regularly.
- Use this as a template for future projects.